Now, after a couple of years of work, PackageKit is well on its way to becoming the standard software manager in the near future and is already the default in Fedora 9 onwards, while other distributions such as SUSE and Ubuntu are adopting it as well.

PackageKit is a tool designed to make installing, removing and updating software easy, and to provide the same graphical interface across multiple distributions. How is this possible? Before we get to that, it is important to understand what PackageKit is *not*. It is not a replacement for the dependency resolvers such as *yum*, *apt* or *zypper*. It does not do any dependency resolution on its own. PackageKit provides neutral interfaces for common functionality, such as installing or removing a package, which is mapped into the distribution-specific backends that take advantage of the native tools already in the distribution to do all the grunt work. The goal: in the near future, all distributions will share the same interfaces and for the most part you don't have to worry about the underlying tools.

Before we move on, let's take a quick look at the graphical interface shown in Figure 1. This one is the GNOME frontend.

PackageKit is a UI-agnostic library. There is KpackageKit under rapid development, which you can easily guess is a KDE frontend to PackageKit. They share the same common library.

As you can see in Figure 1, there is a fairly standard hierarchical view of packages in a Fedora 10 software repository. You can install software as a collection that is quite useful if you want all the packages that form the new LXDE Desktop Environment in one go, for instance. The filtering capabilities are a bit unique. Let me explain that a bit more. You can filter the applications that are shown based on whether they are graphical or non-graphical, for development or regular use, have been installed or not installed, and also whether they are free and open source, or proprietary.

The last part is interesting. As I noted before, the functionality is dependent on the underlying package manager. RPM stores the licensing information within the metadata itself and it is possible to sort and filter, based on this. Other formats like the one used by Debian do not. PackageKit enables or disables parts of the graphical tools automatically, based on whether the underlying backend supports it. So it is entirely possible to have a partially-supported backend in PackageKit and add more support incrementally. If you are a developer of a small distribution with its own unique package manager, writing a backend to hook it up with PackgeKit is much more effective than writing all the tools on your own, and this is exactly what many smaller distributions such as Paldus do, saving them lots of redundant work.

Major unique features

PackageKit is quite sophisticated and takes advantage of a number of new technologies. It integrates with PolicyKit, which allows a very fine-grained security model. I can, for example, give access to update my system to my family members but not let them remove any packages from it. PackageKit also has a daemon that is activated on demand and does not waste system resources. It is also session-aware and doesn't break just because you end your session or have fast user switching to let another user log in.

Despite all this, PackageKit is fundamentally tuned to the basic needs of users and all the features are developed with this in mind. Recently, PackageKit added a number of new interesting features as well. Let's briefly go through the major features that make PackageKit unique, very user friendly and way ahead of other desktop software management tools.

Figure 1: The PackageKit GUI

Figure 2: Do you want to run the newly-installed application(s)?

Figure 3: Updates classified as security, bug fix and enhancement updates

Easy run

When you install an application or a group of applications, PackageKit prompts you to run them. This is quite useful because non-technical users often are not able to find where the newly-installed application can be located. Linux desktop environments usually have a well-categorised menu, but PackageKit makes it even easier and more user-friendly. Refer to Figure 2.